WEEK 2 · LESSON 12 OF 24 · HANDOUT

Week 2 Project: A Customer-Support Assistant

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=EWvNQjAaOHw — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

🎓 **Go deeper:** PyOhio 2025: Demystifying AI Agents with Python Code — https://www.youtube.com/watch?v=jOEdb9HnwsQ

Your goal in this lesson: Build and test a complete LLM-powered support assistant that classifies messages, extracts details, protects personal data, drafts replies from company rules and knows when to hand over to a person.

The brief

PayKwik (a made-up mobile-money company) receives thousands of messages a day. Build an assistant that, for each message:

1. **Removes personal data** (phone numbers, ID numbers) before anything is sent to the AI provider.
2. **Classifies** it (payment, network, account) and marks urgency — as validated structured data.
3. **Drafts a reply** using only the company's rules.
4. **Escalates** to a human when it should.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
support_messages = [
    ("I sent 20,000 RWF to my brother yesterday but he did not receive it", "payment"),
    ("Internet is very slow in Huye since this morning", "network"),
    ("I forgot my mobile money PIN, please help", "account"),
    ("Money was deducted twice when I paid at the supermarket", "payment"),
    ("No 4G signal in my village for two days", "network"),
    ("My SIM card is blocked after I typed the wrong PIN", "account"),
    ("How do I reverse a payment to a wrong number?", "payment"),
    ("The data bundle I bought is not working", "network"),
]
CATEGORIES = ["payment", "network", "account"]
print(len(support_messages), "messages")
```

Part 1 — Protect personal data first

```
import re

PHONE = re.compile(r"\b(?:\+?250|0)7[2389]\d{7}\b")      # Rwandan mobile numbers
NATIONAL_ID = re.compile(r"\b1\d{15}\b")                  # 16-digit national ID format

def redact(text):
    text = PHONE.sub("[PHONE]", text)
    return NATIONAL_ID.sub("[ID]", text)

print(redact("Please send it to 0788123456, my ID is 1199880012345678"))
```

Only send what the AI needs. Keep the original message in your own secure system; the provider sees the redacted version. This follows the "data minimisation" principle in Rwanda's data protection law.

Part 2 — Classify with a validated schema

```
from pydantic import BaseModel
from typing import Literal

class Triage(BaseModel):
    category: Literal["payment", "network", "account"]
    urgent: bool
    confidence: float          # 0 to 1: how sure the model is

triage_tool = {"name": "triage", "description": "Classify a customer support message.",
               "input_schema": Triage.model_json_schema()}

def triage(message):
    response = client.messages.create(
        model=MODEL, max_tokens=200, tools=[triage_tool],
        tool_choice={"type": "tool", "name": "triage"},
        system="You triage messages for a mobile-money company. Mark as urgent anything
involving lost money, fraud or blocked access.",
        messages=[{"role": "user", "content": f"<message>{message}</message>"}],
    )
    data = next(b.input for b in response.content if b.type == "tool_use")
    return Triage.model_validate(data)

print(triage(redact("My SIM is blocked and I cannot receive my salary, number 0788123456")))
```

Part 3 — Draft replies from the rules only

```
RULES = '''PayKwik support rules (practice text):
- Failed or double payments: refunds are checked within 48 hours; give the customer the
transaction ID if they have it.
- Forgotten PIN: the customer must visit a PayKwik agent with their national ID. Never ask for
the PIN.
- Network problems: ask for the location and the time it started; restarting the phone often
helps.
- Never promise a refund amount or date beyond these rules.'''

def draft_reply(message, category):
    return ask(
        f"<rules>\n{RULES}\n</rules>\n\n<message>{message}</message>\n\n"
        f"Write a short, polite reply (at most 3 sentences) to this {category} message. "
        "Use only the rules. If the rules do not cover it, say a colleague will follow up.",
        system="You are PayKwik's support assistant. Be kind and clear.",
        max_tokens=250,
    )
```

Part 4 — Decide when a person takes over

```
ALWAYS_HUMAN = ("fraud", "stole", "steal", "scam", "hack", "police", "court")   # word stems:
"stole" also matches "stolen"

def handle(message):
    safe = redact(message)
    t = triage(safe)
    needs_human = (t.urgent or t.confidence < 0.7
                   or any(word in message.lower() for word in ALWAYS_HUMAN))
    reply = None if needs_human else draft_reply(safe, t.category)
    return {"category": t.category, "urgent": t.urgent, "confidence": round(t.confidence, 2),
            "to_human": needs_human, "reply": reply}

for text, _ in support_messages[:3]:
    print(text[:50], "->", handle(text))
print(handle("Someone stole my phone and is using my mobile money!"))
```

The escalation rules are **code**, not prompt text: they work even if the model is wrong or manipulated.

Part 5 — Test before you trust

```
correct, to_human = 0, 0
for text, expected in support_messages:
    result = handle(text)
    correct += result["category"] == expected
    to_human += result["to_human"]
print(f"category accuracy: {correct}/{len(support_messages)}, sent to a person: {to_human}")
```

Record accuracy, the share sent to humans, average tokens per message and estimated monthly cost. Read 20 drafted replies yourself: are they polite, correct and within the rules?

Part 6 — Report (two pages)

1. The problem, the users and the risks.
2. The pipeline diagram: redact → triage → escalate or draft → (human review).
3. Test results: accuracy, escalation rate, cost estimate.
4. Five example replies with your comments.
5. Data protection: what is sent to the provider, what is stored, for how long.
6. What must be true before going live (for example: a human reviews drafts for the first month).

Marking guide (self-check)

Area	Excellent work
Privacy	Phone numbers and IDs redacted before every API call
Reliability	Structured output validated; errors handled
Safety	Escalation rules in code; no promises beyond the rules
Evaluation	Test set, accuracy and escalation rate reported
Communication	A manager can decide whether to pilot it

Remember this: A useful LLM app is a pipeline around the model — privacy first, validated structure, rules in code, human hand-off and testing — not just a clever prompt.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6. Then add five messages in Kinyarwanda to the test set and measure how the assistant handles them.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why redact phone numbers and ID numbers before calling the AI provider?
   - A. To save tokens only
   - B. Because the model cannot read numbers
   - C. To send only the data the task needs and protect customers' privacy
   - D. To make replies longer
2. Why are escalation rules written in code rather than only in the prompt?
   - A. It is cheaper
   - B. Prompts cannot mention fraud
   - C. Code is shorter
   - D. They then work even if the model is wrong or manipulated
3. Which message should always go to a person?
   - A. Someone stole my phone and is using my mobile money
   - B. How do I check my balance?
   - C. Internet is slow
   - D. What are your hours?
4. Why does the reply prompt include the company rules inside <rules> tags?
   - A. To make the model creative
   - B. To hide them from customers
   - C. Rules are required by the API
   - D. So replies are based only on approved rules
5. What does the confidence field help with?
   - A. Sending uncertain cases to a person
   - B. Formatting
   - C. Pricing
   - D. Translation
6. What should you do before going live?
   - A. Remove the tests
   - B. Test on labelled messages and have people review drafts at first
   - C. Hide the escalation rules
   - D. Nothing

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.